

Binära träd

För att se ytterligare ett exempel på länkade strukturer i C visas här en ADT för binära träd och dess implementation. Kanske ni kommer ihåg att sorterade sökträd är en användning av binära träd. Ett sådant implementeras här. För att variera konfekten lite har jag lagt till en möjlighet att iterera trädet (enlig preorder). Ett sökträd används förstås till att söka efter saker, t.ex. kan man tänka sig att lägga ip-adresser där för att få effektiv sökning då man behöver finna en viss nyckel.

Några trädoperationer:

- En funktion som skapar ett träd. **In:** Inget **Ut:** Ett tomt träd
- En funktion som slänger bort ett träd (analogt med listförstöraren , minnet måste återlämnas). **In:** Ett giltigt träd **Ut:** Inget
- En funktion som sätter in en ny nyckel i ett sorterat binärt sökträd. **In:** Ett giltigt träd, en unik nyckel **Ut:** Ett giltigt träd med en ny nod (innehållande nyckeln) insatt på rätt ställe
- En funktion som svarar om en viss nyckel finns **In:** Ett giltigt träd, nyckeln **Ut:** Ja om nyckeln finns annars nej
- En funktion som skapar en iterator för ett träd, endast en iterator i taget kan användas. **In:** Ett giltigt träd **Ut:** En iterator (pekar till roten)
- En funktion som ger nästa iterator. **In:** En giltig iterator **Ut:** En iterator som representerar nästa nod i trädet
- En funktion som tar bort en viss nod i trädet, efter borttagning är iteratorn inte längre giltig. **In:** Ett giltigt träd, en iterator till trädet **Ut:** Ett giltigt träd med angiven nod borttagen
- En funktion som ger den nyckel som en viss iterator representerar. **In:** En giltig iterator **Ut:** En nyckel

Precis som i fallet med länkade listor kan man tänka sig andra och fler operationer. Gör gärna en egen uppsättning för att öva. Det jag gjort syftar främst till att ge uppslag för hur man kan implementera länkar i C. Jag kommer tillbaks till detta exempel under föreläsning 8 (ett skäl att både operatorer och implementation kanske är lite ovan utformade är att jag behöver ett exempel till att bygga den föreläsningen på :-)

Först prototyperna för operationerna:

- `t_btree *create_tree(void);`
- `void destroy_tree(t_btree *t);`
- `int add_node(t_btree *t, int key);`
- `int exist(t_btree *t, int key);`
- `t_iterator *first(t_btree *t);`
- `t_iterator *next(t_iterator *it);`
- `void remove_it(t_btree *t, t_iterator *it);`
- `int get_data(t_iterator *it);`

Jag tänkte att `add_node` kan få returnera 0 om nyckeln redan fanns och 1 annars.

```
typedef struct t_btree {
    struct t_node *root;
} t_btree;
typedef struct t_node t_iterator;
```

Det borde vara OK så långt. Jag tänkte att vi skulle passa på att använda upp-pekare i trädet också, om ni inte implementerat detta tidigare så bör det ha framgått av litteraturen i den tidigare kursen att sådana kan vara nyttiga. Det ger lite av samma effekt som i fallet med bakåtppekare i länkade listor: det blir en liten aning knöligare när man ska sätta in och ta ut saker, men kan bli lite enklare i några

andra operationer.

```
typedef struct t_node {
    int key;
    struct t_node *left, *right, *up, *next;
} t_node;
```

Sen har det hamnat en next pekare där också. Den behövs för att begränsa kostnaden för att stega fram iteratorn till `ordo(1)`. Den sätts under *iterationen* och ser till att iteratorn går direkt från 'right'-noder (efter att dess barn har genomlöpts) till nästa enligt pre-order (denna ligger av naturliga skäl ovanför). Om du tycker att det verkar knepigt så rita ett träd som du följer medan du tittar på implementationen av 'next'-funktionen. Och så är frågan om detta är meningsfullt, eller bara ett exempel för att leka med pekare? - Det är mycket meningsfullt för vissa implementationer! Du kan gärna jämföra ADT:n med den för länkade listan när det gäller kostnader för olika operationer.

```
/* ----- Local functions: */
static void destroy_nodes(t_node *t)
{
    if (t) {
        destroy_nodes(t->left);
        destroy_nodes(t->right);
        free(t);
    }
}

static t_node *make_node(t_node *parent, int key)
{
    t_node *nd;
    nd = (t_node*)malloc(sizeof(t_node));
    if (nd) {
        nd->key = key;
        nd->left = nd->right = NULL;
        nd->up = parent;
    }
    return nd;
}

static t_node *insert_node(t_node *parent, t_node *nd, int key, int *error)
{
    if (nd == NULL) {
        nd = make_node(parent, key);
        *error = nd == NULL;
    } else if (nd->key > key)
        nd->left = insert_node(nd, nd->left, key, error);
    else if (nd->key < key)
        nd->right = insert_node(nd, nd->right, key, error);
    else
        *error = 1;
    return nd;
}

static t_node *unhook_rightmost(t_node *nd)
{
    if (nd->right) {
        return unhook_rightmost(nd->right);
    } else {
        nd->up->right = nd->left;
        if (nd->left)
            nd->left->up = nd->up;
        return nd;
    }
}
```

```

    }
}

static t_node *find_node(t_node *nd, int key)
{
    if (nd) {
        if (nd->key < key)
            return find_node(nd->left, key);
        if (nd->key > key)
            return find_node(nd->right, key);
    }
    return nd;
}

/* ----- Exported functions: */
t_btree *create_tree(void)
{
    return (t_btree*)calloc(sizeof(t_btree), 1);
}

void destroy_tree(t_btree *t)
{
    destroy_nodes(t->root);
    free(t);
}

int add_node(t_btree *t, int key)
{
    int error;
    t->root = insert_node(NULL, t->root, key, &error);
    return error;
}

int exist(t_btree *t, int key)
{
    return find_node(t->root, key) != NULL;
}

t_iterator *first(t_btree *t)
{
    t->root->next = NULL;
    return t->root;
}

t_iterator *next(t_iterator *it)
{
    if (it->left) {
        if (it->right) {
            it->right->next = it->next;
            it->left->next = it->right;
        } else
            it->left->next = it->next;
        return it->left;
    } else if (it->right) {
        it->right->next = it->next;
        return it->right;
    } else {
        return it->next;
    }
}

void remove_it(t_btree *t, t_iterator *it)
{
    t_node *newroot;

```

```

if (it->left) {
    if (it->left->right == NULL) {
        newroot = it->left;
        newroot->right = it->right;
    } else {
        newroot = unhook_rightmost(it->left);
        newroot->left = it->left;
        newroot->right = it->right;
        if (newroot->left)
            newroot->left->up = newroot;
        if (newroot->right)
            newroot->right->up = newroot;
    }
} else
    newroot = it->right;
if (newroot)
    newroot->up = it->up;
if (it->up) {
    if (it->up->left == it)
        it->up->left = newroot;
    else
        it->up->right = newroot;
} else
    t->root = newroot;
free(it);
}

int get_data(t_iterator *it)
{
    return it->key;
}

```

Ni kommer ihåg hur man tar bort en nod i ett sorterat binärt träd? - Ett sätt att bibehålla den sorterade strukturen är att låta den nod man ska ta bort erstättas av antingen den största i vänster barnträd (det är den som används här ovan) eller den minsta i höger barnträd. Det blir skapligt enkelt men bibehåller inte balansering. Insättningen ovan gör inte heller det.

Nextpekarna som används av nextfunktionen är som sagt till för att med den konstruktion av iterator som valts klarar man att flytta fram iteratorn med kostnaden $\text{ordo}(1)$. I annat fall skulle man behöva rekursera upp till nästa nod som har ett oitererat högerbarn varje gång man kom till ett löv. Next kommer därför att peka på tvären i trädet, alltid på nästa som står i tur, och den sätts under iterationen så snart detta är känt. För små träd kommer den bara att peka från vänster till höger syskon. För större träd kommer pekaren att dras ned så snart detta är möjligt så att pilen pekar snett upp åt höger, pga satserna:

```

it->left->next = it->next;
it->right->next = it->next;

```